

RESEARCH PRESENTATION

CMatrix

*Dual-Graph-Guided LLM-Orchestrated
Multi-Agent Framework for Autonomous
Vulnerability Assessment and Penetration Testing (VAPT)*

Autonomous Pentesting

AI Agent Orchestration

Attack Graph Reasoning

Supervisor Meeting · June 2026

Black-Box · Grey-Box · Network · Web · REST API

THESIS SUPERVISOR

Dr. Md. Abdur Razzaque

Professor & Chairman

Dept. of Computer Science & Engineering
University of Dhaka

PRESENTED BY

Nishan Paul

Reg. 55

Md Rakibur Rahman

Reg. 49

July 2025 · Batch 05

Professional Masters in Information and Cyber Security

What Existing VAPT Systems Get Wrong

✗ No Structured World Model

Systems reason from flat conversation history or task queues — no graph of what the target actually is.

✗ No Attack Path Reasoning

Finished tasks are logged, not understood. Systems cannot represent which vulnerabilities chain together or why.

✗ Fragile Re-Planning

Dynamic re-planning is ad-hoc. There is no formal trigger grounded in discovered evidence.

✗ Arbitrary Termination

Missions end on time-limits or empty queues — not because the attack surface and all chains are truly exhausted.

“

They automate tool execution. CMatrix automates the reasoning of a professional penetration tester.

— CMatrix Design Philosophy



ROOT CAUSE

Existing systems have no structured model of the target environment and no structured model of what attack paths are possible. They know what they did — not what the target is, or what can be done to it.

CMatrix Solution → **Dual-graph world model: ASG captures discovered reality · APG captures inferred attack opportunity**

Academic Papers + Open-Source Repositories That Shaped CMatrix



ACADEMIC PAPERS

1

PentestGPT

USENIX Security '24

Tripartite Reasoning / Generation / Parsing pipeline.
Pentesting Task Tree as long-session context memory.

Gap: Task Tree records tasks performed — not discovered facts. No structured target model; no representation of what can be done to it.

2

AutoAttacker

arXiv '24 — Xu et al.

Modular planner / summarizer / code-generator for post-breach automation.
Experience manager reuses attack subtasks within one mission.

Gap: Experience reuse bounded to a single mission. Every new mission resets to zero knowledge — no cross-mission accumulation.

3

Teams of LLM Agents (HPTSA)

arXiv '24 — Zhu et al.

Hierarchical planner delegates to task-specific sub-agents.
Task-specific document injection improves zero-day exploitation by 2.1x.

Gap: No structured attack graph. No formal representation of which vulnerabilities chain together or how far a chain has been validated.

4

PentestAgent

AsiaCCS '25 — Shen et al.

Planning agent with RAG-backed shared memory.
Execution agent self-diagnoses failed exploits before abandoning.

Gap: RAG memory conflates discovery facts with reasoning hypotheses. Termination on empty task queue — attack surface may be unexplored.

5

VulnBot

arXiv '25 — Kong et al.

Five-module pipeline around a Penetration Task Graph.
Summarizer condenses phase output to prevent history leakage.

Gap: No formal attack graph. Context compaction is ad-hoc. No lifecycle for attack chains: no risk scoring or formal chain resolution.



OPEN-SOURCE REPOSITORIES

6

PentAGI

vxcontrol/pentagi

Production multi-agent pentesting platform.

Adviser detects agent fixation; Reflector nudges stuck agents.

Gap: Fixation detection is reactive — fires after repeated tool calls. No proactive graph-driven surface-exhaustion check.

7

Claude Code

Anthropic — yasasbanukaofficial/claude-code

Two-stage fast-filter + chain-of-thought tool-call permission classifier.

27-event named hook architecture for operator intervention.

Gap: Permission classification is general-purpose — not domain-aware. No offensive risk tier, scope enforcement, or security-intent routing.

8

Hermes Agent

NousResearch/hermes-agent

Closed learning loop: skill crystallization from completed tasks.

Structured trajectory export for SFT/RL training via DSPy + Atropos.

Gap: Skill crystallization and trajectory export are general-purpose. No security-domain indexing, APG integration, or dual-graph event format.

What CMatrix Assesses — and What It Does Not

✓ IN SCOPE

Web Applications

HTTP/HTTPS services, CMS, REST APIs, GraphQL endpoints.

Network Services

Exposed ports, services, TLS/SSL, open protocols.

API Attack Surface

REST endpoints, parameter injection, IDOR, auth bypass.

Black-Box Mode

Zero prior knowledge. Operator provides: root domain + scope declaration only.

Grey-Box Mode

Partial knowledge. Operator additionally provides: known IP ranges, credentials, network topology fragments. ASG Host nodes are pre-seeded.

Authorised Assessments

CMatrix operates only within declared scope. Scope check fires before every tool execution at the Risk Gate.

✗ OUT OF SCOPE

✗ White-Box (source code review, SAST)

✗ Mobile Applications (iOS, Android)

✗ Cloud / IoT Infrastructure (AWS, Azure, embedded devices)

✗ Wireless Networks (Wi-Fi, Bluetooth, RF)

✗ Active Directory / Domain Services

✗ Physical / Social Engineering Attacks

OPERATOR CONFIGURATION (one-time, before mission start)

→ Root domain (e.g. shopvault.io)

→ Scope declaration (all subdomains · web apps · APIs)

→ Assessment mode (Black-Box or Grey-Box)

→ Grey-Box extras if applicable (IP ranges, credentials)

All subsequent tool calls, reasoning, and reporting are fully autonomous — zero manual commands during the assessment.

CMatrix — Three-Tier Architecture Overview

① ORCHESTRATION

OPERATOR

Target · Scope · Mode

COMMANDER AGENT

- Reads ASG + APG state
- Plans & delegates tasks
- Seeds APG AttackChains from Vulnerability nodes
- Approves High-risk ops
- Writes to APG only

VAPT PROTOCOL PROMPT

Phase rules · Re-plan triggers · Termination conditions · Methodology-as-config (C7)

② DUAL-GRAPH WORLD MODEL

ATTACK SURFACE GRAPH (ASG)

Domain

Host

Port

Service

Technology

Endpoint

Parameter

Vulnerability

Evidence

Discovered Reality — confirmed facts only. Never contains hypotheses.

STRICT SEPARATION

ATTACK PATH GRAPH (APG)

AttackChain

ChainStep

Impact

risk_score · validation_status · priority

Inferred Opportunity — attack reasoning only. Never contains raw scan data.

③ SPECIALIZED AGENTS + TOOL ADAPTER LAYER

Recon

- Amass
- httpx
- Nmap

Analysis

- WhatWeb
- Gobuster
- ffuf
- Nuclei
- ZAP

Research

- NVD
- Exploit-DB
- GitHub

Validation

- SQLMap
- Metasploit

Evidence

EyeWitness

Report

Reads ASG + APG

TOOL ADAPTER LAYER

LOW

Passive

Execute immediately

MED

Active

LLM Classifier → exec/escalate

HIGH

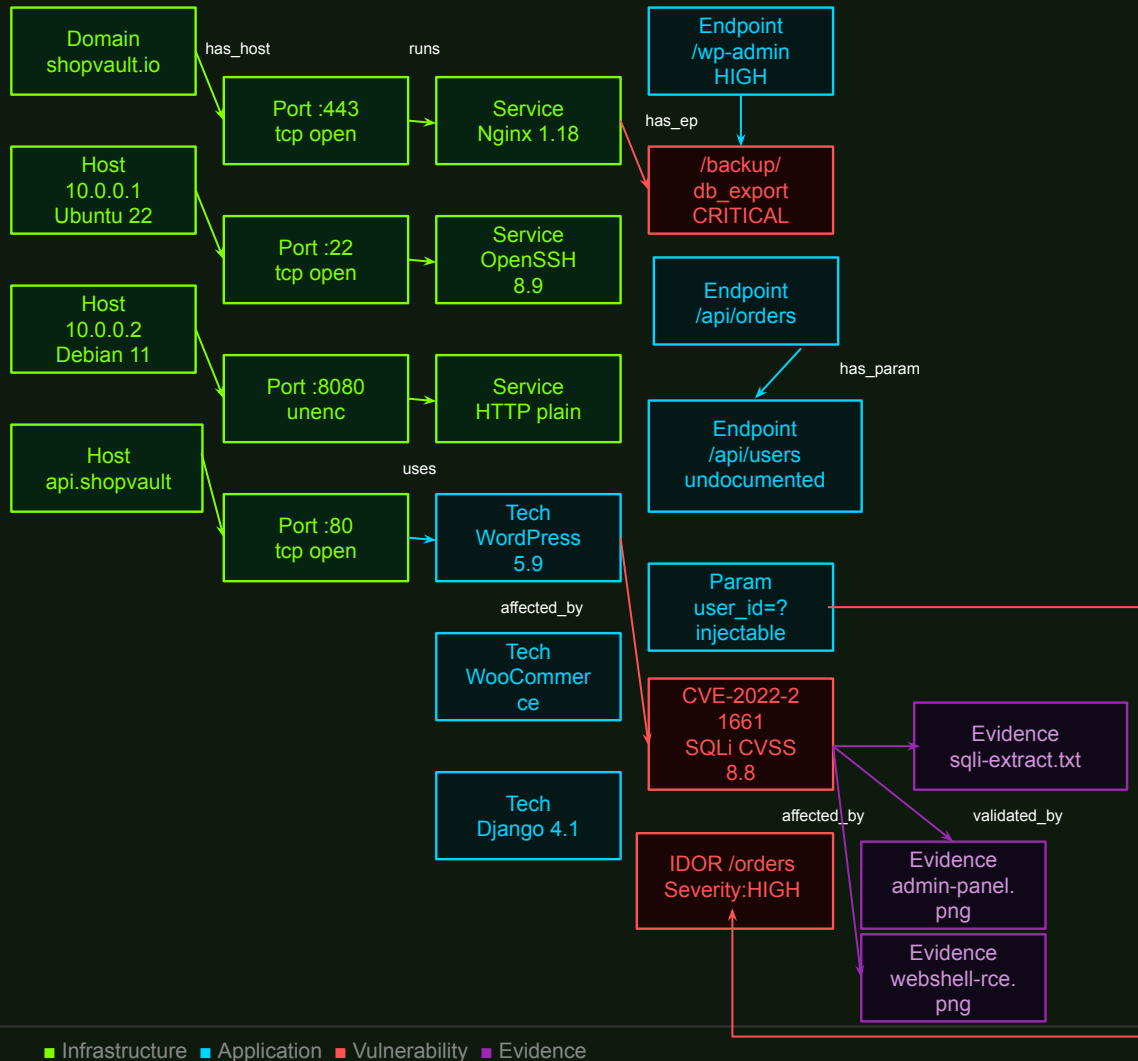
Exploit

Commander Mailbox approval

Two Strictly Separated Knowledge Layers

ASG — Attack Surface Graph

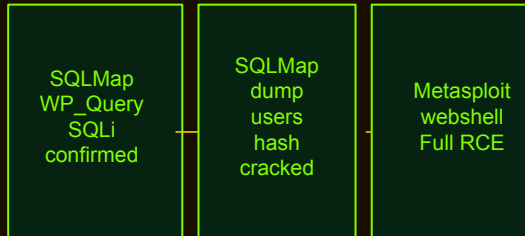
Discovered Reality · Every node = confirmed by tool · Every edge = confirmed relationship



APG — Attack Path Graph

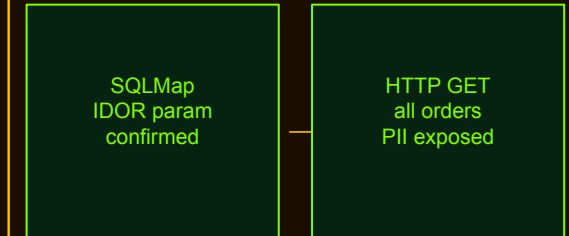
Inferred Opportunity · Commander reasons from ASG vulnerabilities — no raw scan data

Chain-01: SQLi → RCE risk: 9.1 ↑ ✓ VALIDATED



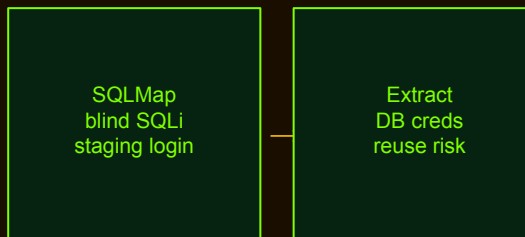
IMPACT CRITICAL · Full RCE · Customer PII exposed

Chain-02: IDOR → Orders risk: 7.5 ✓ VALIDATED



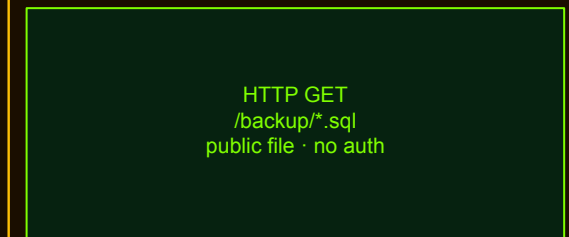
IMPACT HIGH · All orders + PII exposed

Chain-03: Blind SQLi → Creds risk: 8.1 ✓ VALIDATED



IMPACT HIGH · DB credentials extracted · reuse risk

Chain-04: Exposed Backup risk: 7.0 ✓ VALIDATED



IMPACT CRITICAL · Full PII DB download · no exploit needed

APG Priority Queue: #1 Chain-01 · 9.1 · #2 Chain-03 · 8.1 · #3 Chain-02 · 7.5 · #4 Chain-04 · 7.0
· Commander re-ranks on every status change

Separation Principle: Discovery agents write only to the ASG — they never reason about chains. The Commander writes only to the APG — it never runs tools. No agent conflates discovered facts with hypothesised attack reasoning.

Context-Isolated Agents — Spawn · Execute · Return · Discard

CONTEXT ISOLATION MODEL

COMMANDER AGENT

Reads ASG + APG → decides next action

spawn with scoped context

ISOLATED AGENT CONTEXT (fresh per task)

ASG SLICE

Only nodes relevant to this task

APG SLICE

Relevant AttackChains (if any)

TOOL SET

Authorized tools only — no others

TASK SPEC

Commander's current plan item

RETURNS: Structured ASG Delta (new nodes + edges only)

Working context discarded — no raw history passes to Commander

✓ Commander context stays clean

✓ Agents can't contaminate each other

✓ High-risk refusals don't bias planning



Commander

ORCHESTRATION

Reads the dual graph. Plans. Delegates. Never runs tools.

No tools — reasons over ASG + APG

Only agent that writes to APG. Approves High-risk ops via mailbox.



Analysis

PHASE 2

Deep enumeration and vulnerability discovery.

WhatWeb · Gobuster · ffuf · Nuclei · ZAP

Writes Technology, Endpoint, Parameter, Vulnerability nodes.



Validation

PHASE 3

Proves vulnerabilities are real and exploitable.

SQLMap · Metasploit

Self-debugging loop: Diagnose → Contextualize → Adapt → Cap (x3).



Recon

PHASE 1

External reconnaissance and host discovery.

Amass · httpx · Nmap

Writes Domain, Host, Port, Service nodes to ASG.



Research

INTELLIGENCE

Live CVE grounding — only agent with outbound internet access.

NVD · Exploit-DB · GitHub · Vendor Advisories

Enriches ASG Vulnerability nodes with real-time CVSS + PoC data.



Evidence

PHASE 3

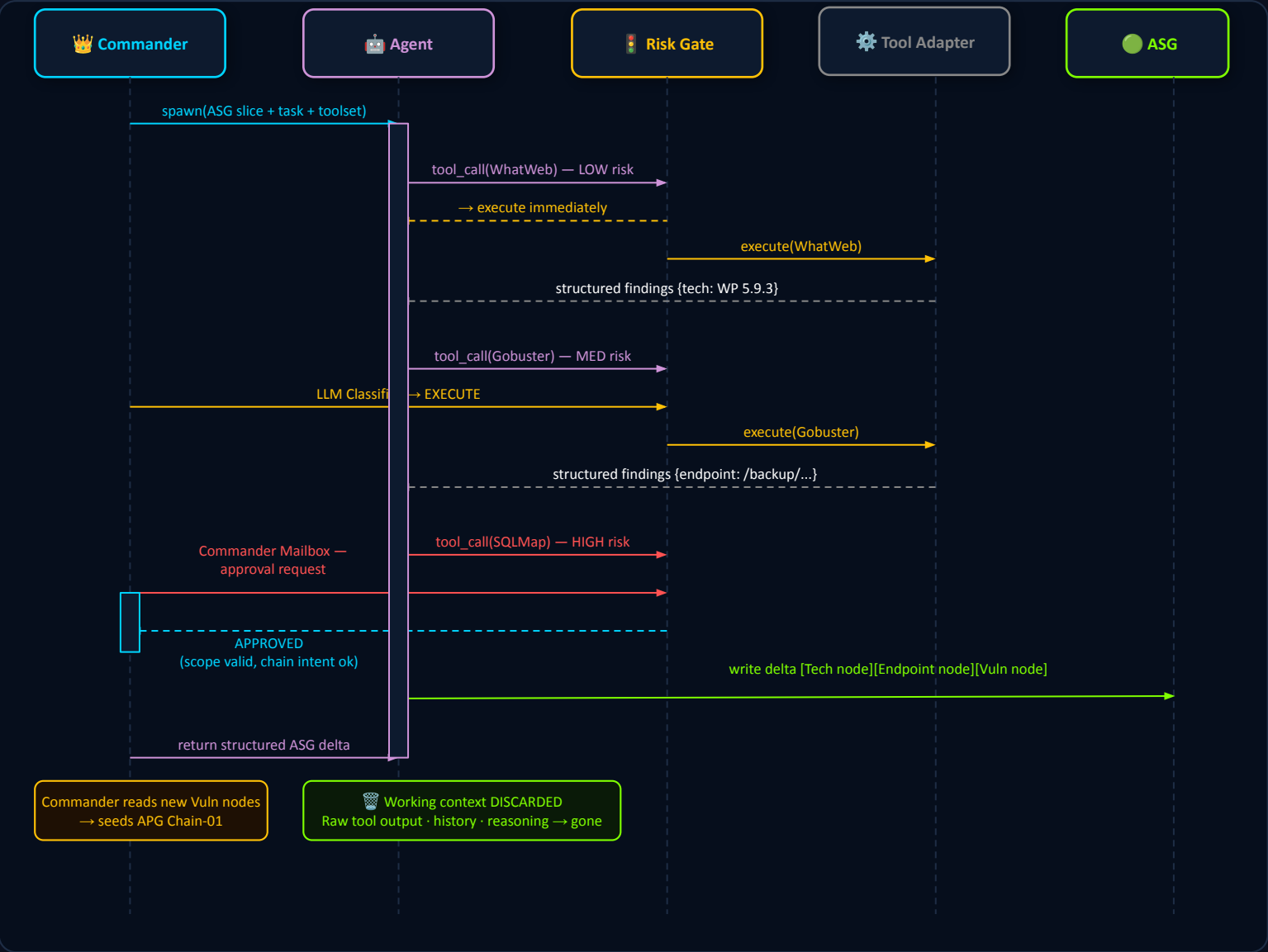
Captures proof artifacts for every validated finding.

EyeWitness

Links Evidence nodes to ASG findings via validated_by edges.

Context-Isolated Agent Spawn Lifecycle

Every agent is born fresh · does exactly one job · dies clean · leaves only structured graph state behind



Agent Spawn Package

- ASG SLICE** Only nodes relevant to this task — not the full graph
- APG SLICE** Relevant AttackChains only (Validation tasks)
- TOOL SET** Authorized tools only — no others available
- TASK SPEC** Commander's current plan item — exact objective
- KNOWLEDGE DOCS** Vuln-class expert docs (Analysis + Validation)

Returns: Structured ASG Delta (new nodes + edges only)

Raw tool output · conversation history · intermediate reasoning → DISCARDED

✓ Commander Stays Clean

Commander only ever sees ASG/APG state — never thousands of lines of raw tool output. Reasoning context stays surgically focused.

✓ Agents Can't Contaminate

Agent A's verbose history never appears in Agent B's context. Knowledge passes only through the ASG. No shared memory. No cross-pollution.

✓ Rejections Don't Bias Planning

When Commander rejects a High-risk tool call, that rejection never appears in the Commander's own context. Refusals don't accumulate and skew future decisions.

🎯 Result: Long missions with many agents produce the same reasoning quality as single-agent tasks — context quality does not degrade with mission complexity.

11 Industry-Standard Security Tools — AI Agent Operated

PHASE 1 · RECONNAISSANCE

1

Amass

Subdomain Enumeration · Recon Agent

DNS brute-force · certificate transparency logs · passive OSINT.
Discovers all external subdomains and maps attack surface entry points.

2

httpx

Live Host Probing · Recon Agent

Validates which discovered hosts respond over HTTP/HTTPS.
Identifies web servers, status codes, redirects, TLS details.

3

Nmap

Port + Service Scan · Recon Agent

Port scanning, service fingerprinting, OS detection. Optional
NSE script execution for vuln detection on open services.

PHASE 2 · ANALYSIS

4

WhatWeb

Technology Fingerprinting · Analysis Agent

Identifies CMS, frameworks, server software, JS libraries, and versions
from HTTP responses. Seeds Technology nodes for the ASG.

5

Gobuster

Directory Brute-Force · Analysis Agent

Discovers hidden paths, admin panels, backup files, and exposed
resources via wordlist-based HTTP path enumeration.

6

ffuf

API + Parameter Fuzzing · Analysis Agent

Fast web fuzzer for API route discovery, parameter fuzzing, virtual host
enumeration, and undocumented endpoint identification.

7

Nuclei

Template-Based Vuln Scan · Analysis Agent

Matches discovered services and technologies against a library of CVE and
misconfiguration templates for automated vulnerability detection.

8

OWASP ZAP

Active Web App Scan · Analysis Agent

Crawls and actively tests for OWASP Top 10: XSS, CSRF, injection flaws,
authentication weaknesses, and insecure direct object references.

PHASE 3 · VALIDATION

9

SQLMap

SQL Injection Validation · Validation Agent

Automated SQLi detection and exploitation. Confirms
injection points, extracts database data, tests for OS-level
access escalation.

10

Metasploit

Exploit + RCE Demo · Validation Agent

Executes known exploits against identified vulnerabilities.
Validates APG ChainSteps, demonstrates impact, achieves
proof of exploitation.

11

EyeWitness

Screenshot Evidence · Evidence Agent

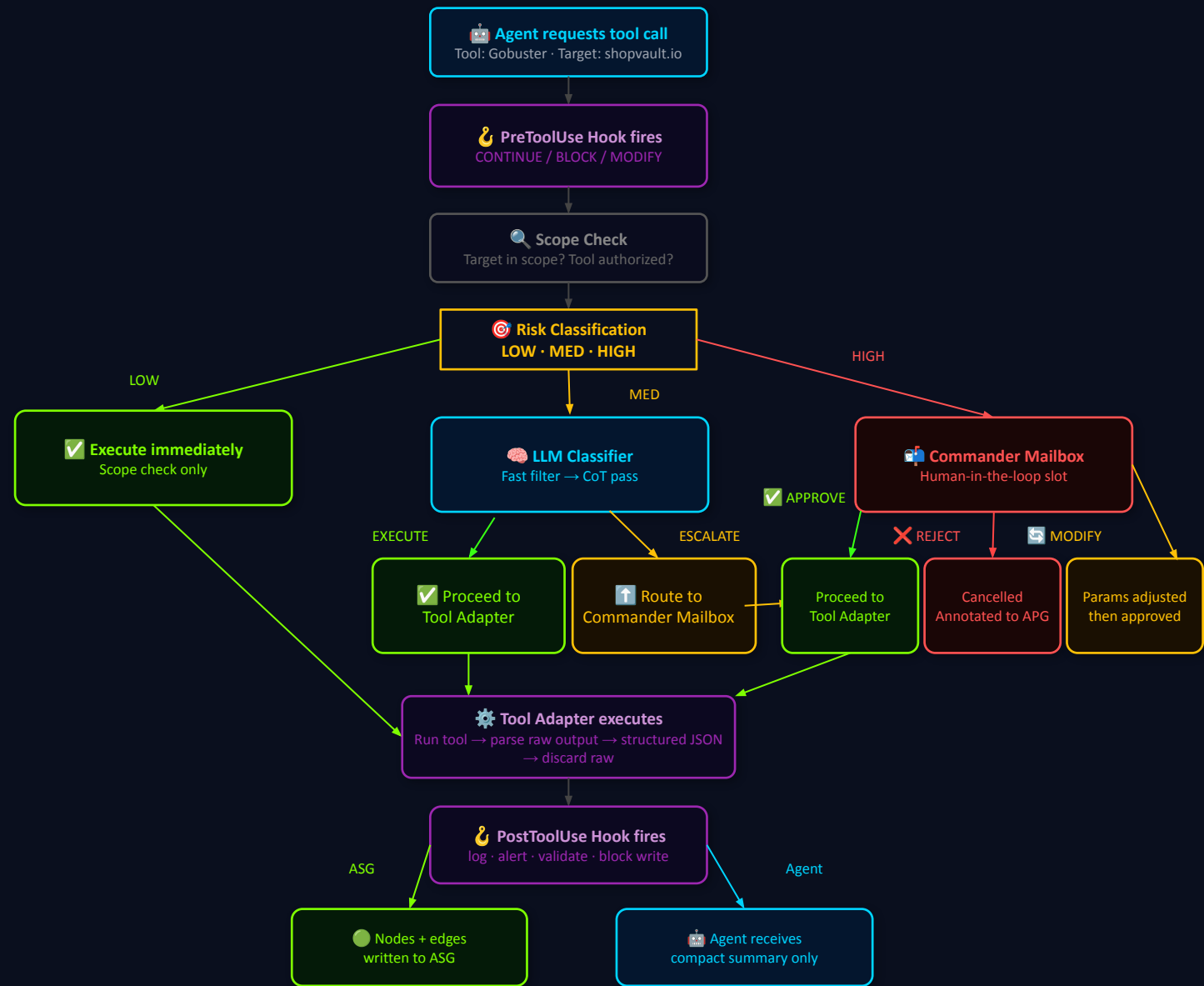
Headless screenshot capture of web pages, exposed panels,
and API responses. Produces visual proof artifacts linked to
ASG Evidence nodes.



Every tool is wrapped in a Tool Adapter — agents reason about targets, not command syntax. No irreversible offensive operation executes without Commander-level scope validation.

Tool Risk Gate — Every Tool Call's Journey

No tool executes without passing this gate · No irreversible operation runs without Commander-level scope validation



🧠 LLM Permission Classifier — Medium-Risk Calls

Fast Filter (instant)
Obviously safe? → EXECUTE immediately
Obviously risky? → ESCALATE immediately

Axis 1: SCOPE ALIGNMENT Is the target in declared assessment scope?

Axis 2: CHAIN INTENT Is this call consistent with the active APG AttackChain?

Axis 3: PARAMETER SAFETY Do params exhibit prompt injection patterns or scope drift?

✅ EXECUTE

⬆️ ESCALATE

Tool Risk Tier Summary

🟢 LOW Scope check only
Amass · httpx · WhatWeb · EyeWitness

🟡 MED LLM Classifier → exec/esc
Nmap · Gobuster · ffuf · Nuclei · ZAP

🔴 HIGH Commander Mailbox approval
SQLMap · Metasploit

🔒 Critical Safety Property
No irreversible offensive operation executes without Commander-level scope validation. No Medium-tier call executes without LLM classifier approval.

Commander Loop — Graph-Grounded, Event-Driven, Self-Terminating

Core Planning Loop — Commander runs this every iteration



What Triggers a Re-Plan

ASG Trigger Events

- New Vulnerability node → seed APG chain?
- New Technology node → spawn Research Agent
- New Endpoint node → probe with Analysis Agent

APG Trigger Events

- Chain → PARTIALLY_VALIDATED → re-rank all
- Chain → VALIDATED → pursue next chain
- Chain → RULED_OUT → remove, re-prioritize

Cycle Guard Events

- Same tool call repeated ×3 → Force re-plan
- Repeated different failures → Reflector

Dual Termination Condition

ASG Exhausted
Every node investigated
by appropriate agent

AND
 APG Resolved
All chains **VALIDATED**
or **RULED_OUT**

BOTH TRUE → Report Agent spawned → **MISSION COMPLETE**

Timer-based systems stop mid-chain. CMatrix requires dual-graph resolution.

Every Commander decision traces to a graph event. Every re-plan is triggered by a node write. Mission ends only when the graph demands it — not when a timer fires.

Dual Termination & Context Compaction

DUAL TERMINATION CONDITION — *why both must be true*

ASG exhausted?

Every Domain, Host, Port, Service, Technology, Endpoint, and Parameter node has been investigated by the appropriate agent.

APG resolved?

Every AttackChain sits in a terminal state — VALIDATED or RULED_OUT. None left HYPOTHESIZED or PARTIALLY_VALIDATED.

✗ ASG only — NOT DONE

Surface fully explored but chains still open. Attack reasoning is unfinished.

✗ APG only — NOT DONE

All chains resolved, but new ASG nodes were just written — they might seed new chains.

✗ Neither — NOT DONE

Both conditions incomplete. Mission continues.

✓ Both true — MISSION COMPLETE

ASG fully mapped, every attack opportunity proven or disproven. Report Agent spawned.

Why existing systems fail: timer-based execution stops mid-chain; task-queue-based systems can't express APG resolution at all. CMatrix is the only architecture that evaluates both conditions simultaneously.

CONTEXT COMPACTION — *how long missions stay sharp*

NORMAL OPERATION

MicroCompact — every tool call

Raw tool output is discarded immediately; the agent sees only a 3-line summary.

context grows ↓

AUTOCOMPACT · @ 60% context

Older turns summarized by a scoped LLM call

The summary replaces raw conversation turns. The agent continues uninterrupted.

context grows ↓

FULLCOMPACT · @ 85% context

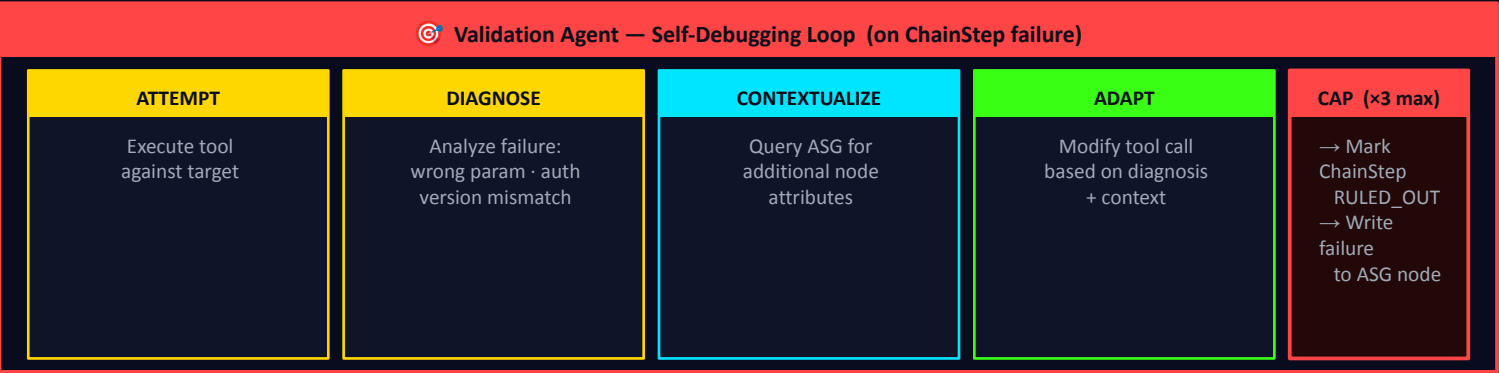
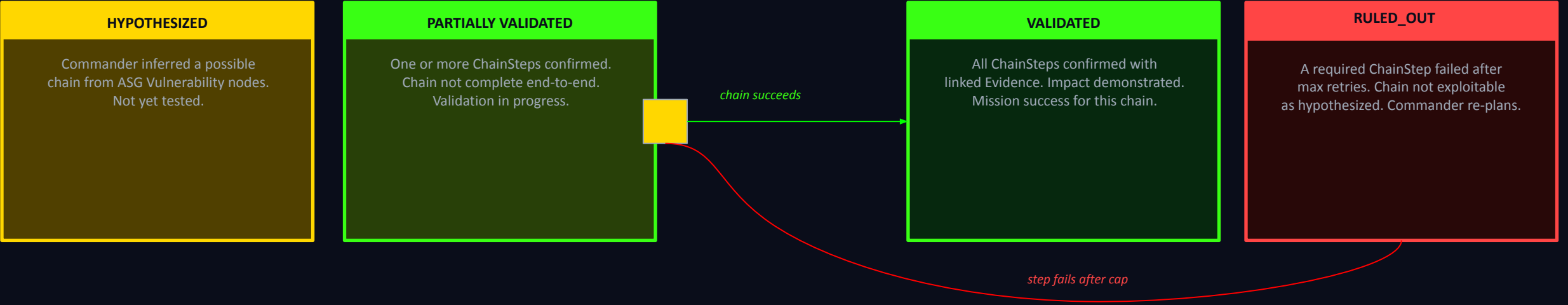
Entire history rebuilt from scratch

Reconstructed from: current ASG snapshot · current APG priorities · last N tool results. Zero intelligence lost — everything important already lives in the graph.

fresh context → loops back to Normal Operation ↺

The ASG is the key: conversation history is expendable because every discovery lives permanently in the graph. That's what makes FullCompact safe — nothing important is ever lost.

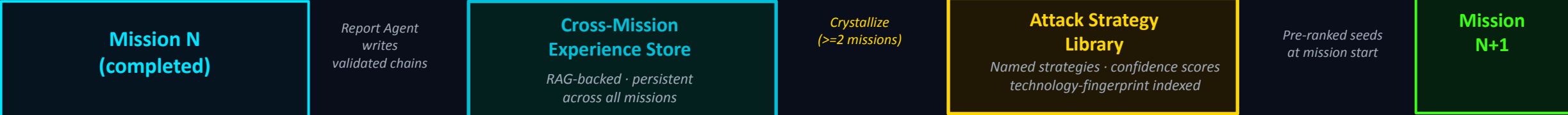
APG Chain State Machine + Validation Agent Self-Debugging Loop



retry (up to cap)

Risk Score + Chain Priority		
risk_score = CVSS × Exploitability × Impact		
Priority = rank across all HYPOTHESIZED + PARTIALLY_VALIDATED chains. Commander pursues highest-priority chain first — re-ranks on every status change.		
shopvault.io — APG Chain Priority Order		
1	Chain-01 — CVE-2022-21661 SQLi → RCE	9.1
2	Chain-03 — Blind SQLi on staging → DB creds	8.1
3	Chain-02 — IDOR on /api/v1/orders → PII	7.5
4	Chain-04 — Direct DB backup download → PII	N/A

Experience Store + Attack Strategy Library — CMatrix Gets Smarter Each Mission



Cross-Mission Experience Store

What it stores

Per-mission · per-chain raw records of validated exploitation outcomes.
Target technology fingerprint · CVE · successful tool invocation · ChainStep sequence · mission outcome.

When it's written

Report Agent writes one entry for every chain with terminal status **VALIDATED** at mission close.

When it's read

Commander queries immediately after Recon Agent writes first Technology nodes to ASG — before Analysis begins. Retrieved records become candidate chain hypotheses.

Research origin

Generalizes AutoAttacker's within-mission experience-reuse mechanism to cross-mission scope. AutoAttacker reuses subtasks within one session; CMatrix accumulates across every mission ever run.

Attack Strategy Library

What it stores

Generalized attack procedures distilled from multiple validated outcomes against the same target technology class. Named strategies (e.g. STRAT-WP-SQLI-001) with confidence scores.

Crystallization threshold

Same technology fingerprint (e.g. WordPress 5.x + WooCommerce + Nginx) produces validated AttackChain in 2 or more independent missions → Commander triggers crystallization.

When it's used

Retrieved at mission start. Strategies are injected as pre-ranked APG AttackChain seeds — prioritized above zero-prior chains because they carry validated track record.

Why it matters

No existing autonomous VAPT system accumulates and generalizes validated exploitation procedures across sessions. Every prior system resets to zero knowledge on each mission. CMatrix's library makes it measurably more efficient on repeat target-type engagements.

Mission: shopvault.io — Black-Box Assessment — Zero Manual Commands

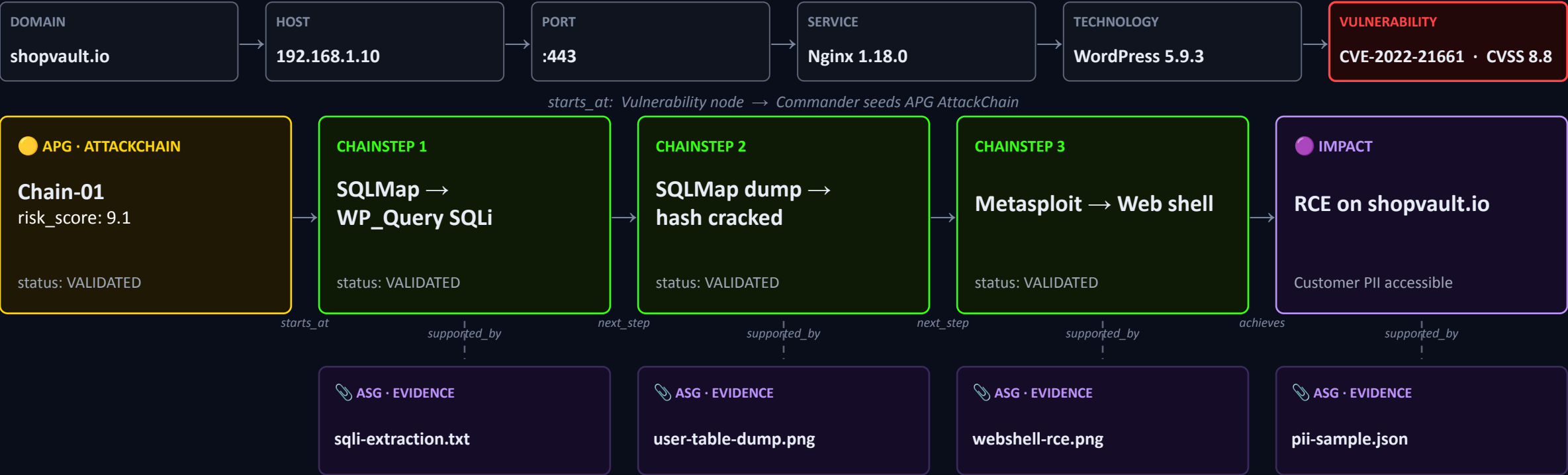
Scope: all subdomains · web applications · REST APIs of shopvault.io · Mode: Black-Box (zero prior knowledge) · Operator configures root domain + scope → presses start.



Chain-01 Full Traceability — From Recon to Evidence

Every claim in the final report follows this exact path through the dual graph — nothing is ever asserted without evidence.

ASG · DISCOVERY — RECON TO VULNERABILITY



Reading this diagram: start at the recon chips (ASG facts) → follow starts_at to the Chain (APG reasoning) → step through the ChainSteps in order → arrive at the Impact → follow supported_by back to the Evidence files (ASG proof).

8.8 → 9.1

risk_score escalated

6

recon-to-CVE hops traced

3

VALIDATED ChainSteps

4

Evidence artifacts linked

0

manual commands

What Makes CMatrix a Research Contribution

First autonomous VAPT system to maintain two strictly separated, continuously evolving graph structures - one for discovered reality, one for inferred attack opportunity.

C1 Dual-Graph World Model

ASG (discovered reality) & APG (inferred opportunity) are strictly separated with enforced write ownership.

C2 Graph-Driven Re-Planning

Re-planning fires on explicit graph triggers (e.g., new vuln node seeds a chain) rather than arbitrary schedules.

C3 APG Attack Chain Lifecycle with Evidence Traceability

Attack chains are first-class entities: risk-scored, prioritized, lifecycle-tracked. Every ChainStep links to ASG Evidence via supported_by.

C4 ASG-Aware Parallel Dispatch

Dependency-safe concurrent tool execution using the ASG itself as the dependency graph, instead of a task scheduler.

C5 Tool Risk Gate + Mailbox

Every tool call is risk-classified before execution. High-risk ops route to Commander for human-in-the-loop approval.

C6 Lossless Context Compaction

ASG-backed three-layer compaction scheme reduces conversation history to near-zero without losing discovered findings.

C7 Methodology-as-Config

Commander's planning policy is encoded as a versioned natural-language document, allowing methodology benchmarking.

C8 Dual-Graph Termination

Mission completion is formally defined as the conjunction of ASG exhaustion (no unexplored nodes) and APG resolution.

C9 Live CVE Grounding

Real-time vulnerability enrichment and exploit feasibility research via a scoped agent, written back into the ASG.

C10 Cross-Mission Experience

Persistent RAG-backed store of validated exploitation outcomes across missions to seed APG hypotheses at mission start.

C11 Attack Strategy Library

Crystallizes validated chains into parameterized strategies when the same target fingerprint succeeds across missions.

C12 Trajectory Export Dataset

Every mission produces a machine-readable decision log capturing ASG/APG triggers, rationale, and actions taken.

How We Will Validate CMatrix Claims Empirically

EVALUATION PLATFORMS

HackTheBox (HTB)

Retired machines — Linux + Windows — known ground-truth vulnerabilities for coverage scoring.

TryHackMe (THM)

Guided CTF environments with intentionally vulnerable web applications and APIs.

Custom Lab VMs

Controlled WordPress + Django stack mirroring shopvault.io for repeatable A/B testing.

RESEARCH QUESTIONS

RQ 1 Does the dual-graph architecture improve vulnerability coverage compared to a flat-memory baseline?

RQ 2 Does formal dual termination reduce false-early-stops vs timer-based termination?

RQ 3 Does cross-mission learning measurably reduce time-to-finding on repeat target-type engagements?

RQ 4 Does the risk gate reduce unintended offensive actions without significantly reducing coverage?

EVALUATION METRICS

Vulnerability Coverage % of known vulns discovered vs ground truth per machine

Chain Precision % of APG AttackChains that result in VALIDATED status

Time-to-Finding Wall-clock time from mission start to first VALIDATED chain

False Early-Stop Rate % of missions terminated before surface exhaustion

Tool Call Efficiency Mean tool calls to first VALIDATED chain per target

BASELINES

✕ PentestGPT: Task-tree memory, no ASG/APG graph split → tests RQ1

✕ VulnBot: Single unified PTG, no surface/path separation → tests RQ1

✕ PentestAgent: RAG-retrieval memory, no cross-mission learning → tests RQ3

ABLATION STUDY

A1 — No dual-graph CMatrix with flat history only (remove ASG + APG)

A2 — ASG only CMatrix with ASG discovery but no APG reasoning

A3 — No compaction CMatrix with unlimited raw history, no lossless context compaction

A4 — No learning CMatrix without cross-mission experience and attack strategy library

CMatrix — What We Built, What We Claim, What We Need

WHAT CMATRIX IS

A dual-graph-guided, LLM-orchestrated multi-agent framework for autonomous Vulnerability Assessment and Penetration Testing. It maintains two strictly separated graph structures — the ASG (what the target is) and the APG (what can be done to it) — and uses both to plan, dispatch, validate, and formally terminate a complete VAPT engagement without human intervention.

CURRENT STATUS

We are currently in the architecture proposal phase, prior to implementation.

- All 12 contributions are fully specified at the architecture level - no implementation has begun.
- Some contributions are more load-bearing than others - prioritization hasn't been finalized.
- HTB/THM evaluation plan is outlined but not yet run.
- This meeting is seeking scope and direction sign-off before implementation work starts.

12 / 12

Contributions specified

0 / 12

Contributions implemented

WHAT WE CLAIM

- CMatrix is the first autonomous VAPT system to maintain two strictly separated, continuously evolving graph structures for simultaneous discovery and attack reasoning.
- Formal dual termination provides a theoretically sound, verifiable completion criterion that prior systems cannot express.
- Cross-mission learning accumulates validated exploitation outcomes across all missions — a capability no prior published VAPT system demonstrates.
- The architecture produces a machine-readable decision trace usable for training security-specialized LLMs.

WHAT WE ARE ASKING FOR

Feedback on Scope

Are the 12 contributions appropriately scoped for our thesis?

Evaluation Guidance

Is the HTB/THM ablation plan sufficient to prove the core claims?

Implementation Priority

Which of the 12 contributions should be fully implemented vs theoretically validated?

Publication Venue

USENIX Security / IEEE S&P / ACSAC — which tier matches the contribution level?